

Writeup — Bypass Me (LLDB + SUID)

picoCTF · Bypass Me · Reverse Engineering (Medium)

Flag : `picoCTF{d3bugg3r_p0w3r_is_4w3s0m3_30b6c610}`

🔍 Résumé de la faille

Un binaire `bypassme.bin` demande un mot de passe. Le mot de passe correct est **décodé en mémoire au tout début de l'exécution** (avant même l'affichage du prompt), via une fonction `decode_password()`. En utilisant un débogueur (LLDB) pour arrêter le programme juste après ce décodage et lire la mémoire, on récupère le mot de passe en clair sans jamais avoir à le deviner ou le casser.

Une seconde subtilité s'ajoute : le binaire a le bit **SUID** actif (appartient à `root`), ce qui lui permet de lire un fichier flag normalement inaccessible à l'utilisateur courant — mais ce mécanisme est désactivé quand le programme tourne sous un débogueur, ce qui oblige à relancer le binaire "normalement" une fois le mot de passe en poche.

🔧 Méthodo — les étapes

1. Connexion et reconnaissance de base

```
ssh ctf-player@foggy-cliff.picoctf.net -p 64701
# mot de passe donné dans l'énoncé
```

Piège de syntaxe évité : `ssh` utilise `-p` (minuscule) pour le port, séparé de l'hôte par un espace — pas `hote:port` comme dans un navigateur.

```
file bypassme.bin
strings bypassme.bin | grep -i pass
```

2. Explorer les symboles du binaire (avant tout désassemblage)

```
lldb ./bypassme.bin
(lldb) image dump symtab bypassme.bin
```

Repérage des fonctions "maison" (pas issues de la libc), qui donnent la structure du programme :

```
sanitize(char const*, char*)
main
auth_sequence()
decode_password(char*)
type_out(char const*, unsigned int)
intro_sequence()
```

`decode_password(char*)` est le nom le plus révélateur : il suggère que le mot de passe correct n'est pas stocké en clair dans le binaire, mais encodé, puis décodé au runtime — exactement le genre de moment qu'un débogueur permet d'intercepter.

3. Premier essai (erroné) : breakpoint sur `auth_sequence`

```
(lldb) breakpoint set --name auth_sequence
(lldb) run
```

Le breakpoint n'a jamais été atteint (`hit count = 0`) alors que le programme s'est exécuté jusqu'au bout (3 tentatives, échec). Leçon : `auth_sequence()` n'est appelée **que si le mot de passe est correct** — elle n'était donc jamais atteinte avec un mauvais mot de passe.

4. Désassembler `main` pour comprendre la logique complète

```
(lldb) disassemble --name main
```

Points clés repérés dans l'assembleur :

```
callq 0x1333 ; decode_password <- appelée TOUT AU DÉBUT, avant l'intro
callq 0x14c6 ; intro_sequence
...
callq 0x1180 ; strcmp (comparaison input vs mot de passe décodé)
testl %eax, %eax
jne 0x1801 ; si DIFFÉRENT -> Access Denied
callq 0x1457 ; si ÉGAL -> auth_sequence()
```

Ça confirme : `decode_password` s'exécute avant tout affichage, le résultat est stocké en mémoire à l'adresse `-0x110(%rbp)` (repérée dans le code assembleur juste avant l'appel à `strcmp`). - La comparaison se fait via `strcmp` entre l'input sanitizé de l'utilisateur et cette valeur décodée. - `auth_sequence()` n'est atteinte qu'en cas d'égalité.

5. Breakpoint fiable : par fichier + ligne (pas par adresse absolue)

Une première tentative avec `--address 0x555555555759` (adresse absolue lue dans le désassemblage) a échoué : breakpoint "unresolved". Cause probable : **ASLR** (Address Space Layout Randomization) qui randomise les adresses mémoire à chaque exécution — une adresse absolue notée lors d'une session n'est pas fiable pour la suivante.

Solution robuste : cibler par fichier source + numéro de ligne, que LLDB résout dynamiquement au runtime :

```
(lldb) breakpoint set --file bypassme.c --line 82
(lldb) run
```

→ Le programme s'arrête juste après le retour de `decode_password()`, avant même l'affichage de l'intro.

6. Lire la mémoire pour extraire le mot de passe décodé

```
(lldb) memory read --format s $rbp-0x110
0x7ffe0068a9e0: "SuperSecure"
```

Mot de passe décodé récupéré directement en mémoire : `SuperSecure` .

7. Piège : le SUID est désactivé sous débogueur

En tapant `SuperSecure` pendant que le programme tournait encore sous LLDB, l'authentification a réussi ("Authenticating...") mais :

```
Flag file not found.
```

Investigation des strings du binaire, qui révèle le chemin recherché :

```
../../root/flag.txt
```

Vérification :

```
ls -la /root/flag.txt
# Permission denied
```

Vérification des permissions du binaire lui-même :

```
ls -la ~/bypassme.bin
-rwsr-xr-x 1 root root 21672 ... bypassme.bin
```

Le `s` à la place du `x` (droits propriétaire) = **bit SUID actif**, binaire appartenant à `root` . Quand il s'exécute normalement, il tourne avec les droits de `root` — d'où sa capacité à lire `/root/flag.txt` malgré les permissions restreintes de `ctf-player` .

Mais le noyau Linux désactive le SUID quand un programme tourne sous un débogueur (mesure de sécurité empêchant un débogueur d'abuser des privilèges élevés d'un binaire SUID). C'est pourquoi le flag n'apparaissait pas pendant la session LLDB.

8. Solution finale : relancer SANS débogueur

```
(lldb) quit
```

```
./bypassme.bin
# Enter password: SuperSecure
```

```
→
Authenticating...
🚩 Flag: picoCTF{d3bugg3r_p0w3r_is_4w3s0m3_30b6c610}
```

🔧 Commandes LLDB utilisées — récap

Commande	Rôle
<code>lldb ./binaire</code>	Lancer LLDB sur un exécutable
<code>image dump symtab binaire</code>	Lister tous les symboles/fonctions du binaire
<code>disassemble --name FONCTION</code>	Voir l'assembleur d'une fonction précise
<code>breakpoint set --name FONCTION</code>	Breakpoint sur l'entrée d'une fonction
<code>breakpoint set --file F --line N</code>	Breakpoint fiable par fichier/ligne (résiste à l'ASLR)
<code>breakpoint list</code>	Lister les breakpoints actifs
<code>breakpoint delete N</code>	Supprimer un breakpoint
<code>run</code>	Lancer/relancer le programme
<code>next / step</code>	Avancer ligne par ligne (sans/avec entrer dans les sous-fonctions)
<code>finish</code>	Terminer la fonction courante et revenir à l'appelant
<code>memory read --format s ADRESSE</code>	Lire une chaîne de caractères en mémoire
<code>x/s ADRESSE</code>	Alternative pour lire une chaîne en mémoire

✔ Réflexes à retenir

- `file` / `strings` / **lecture des symboles** (`image dump symtab`) en tout premier, avant de désassembler quoi que ce soit — ça oriente directement vers les fonctions intéressantes.
- Le nom des fonctions est un indice énorme** : `decode_password` annonçait clairement où chercher.
- Toujours privilégier les breakpoints par fichier/ligne plutôt que par adresse absolue** — l'ASLR randomise les adresses à chaque exécution, seuls les breakpoints "symboliques" (nom de fonction, fichier+ligne) sont fiables d'une session à l'autre.
- Une fonction jamais atteinte** (`hit count = 0`) **est une info en soi** : ça veut dire qu'elle n'est appelée que sous certaines conditions (ici, seulement en cas de succès de l'authentification).
- Le bit SUID** (`s` dans les permissions) **donne des droits élevés à un programme**, mais ce mécanisme est désactivé sous débogueur par sécurité — un binaire qui se comporte différemment sous LLDB que lancé normalement peut être le signe d'un SUID en jeu.
- `ls -la` sur un binaire donne des informations de sécurité précieuses (propriétaire, bit SUID) — un réflexe à avoir sur tout exécutable suspect, pas seulement en reverse.

🔧 Dans la vraie vie

Le pattern "secret décodé en mémoire au runtime, jamais en clair dans le binaire au repos" est une vraie technique **anti-reverse** utilisée par certains logiciels (et malwares) pour compliquer l'analyse statique. Elle retarde l'analyse mais ne l'empêche jamais totalement : un débogueur permet toujours d'observer l'état réel du programme à un instant T, quelle que soit la protection statique mise en place — c'est le principe même de l'analyse dynamique.

Le bit SUID est un mécanisme Linux légitime (ex: `passwd` l'utilise pour modifier un fichier système protégé), mais c'est aussi une source classique d'escalade de privilèges en pentest : un binaire SUID mal sécurisé (permettant d'injecter des commandes, de lire des fichiers arbitraires...) est une des premières choses qu'un pentester cherche lors d'un audit d'élévation de privilèges Linux (`find / -perm -4000 2>/dev/null` pour lister les binaires SUID d'un système).